

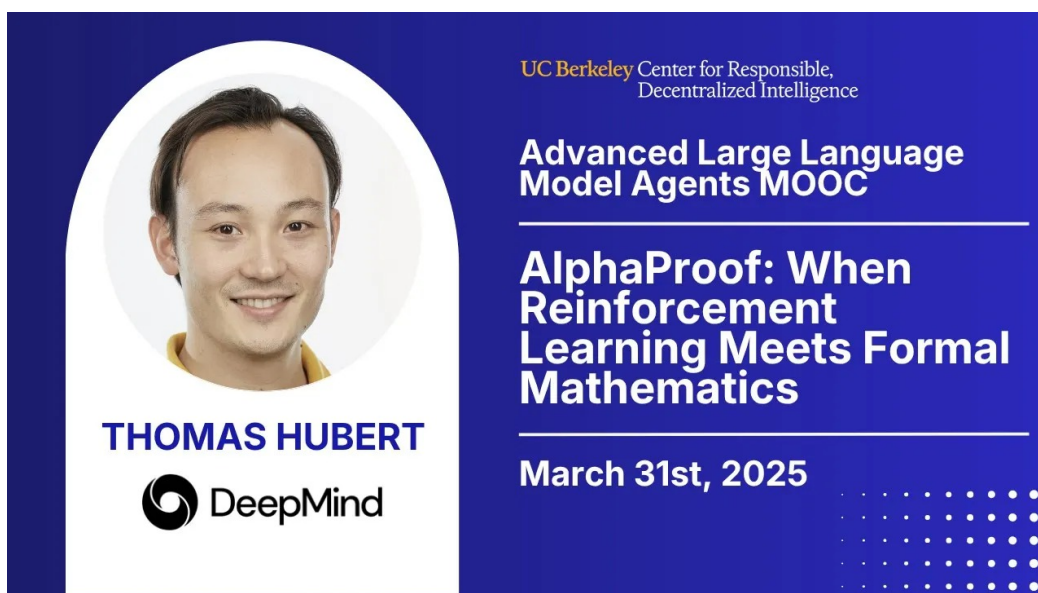
课程讲义

AlphaProof: when reinforcement learning meets formal mathematics

CS294/194-280: Advanced Large Language Model Agents

Thomas Hubert, Google DeepMind

中文教材化讲义 / Codex Harness Build



The banner features a blue background. On the left, there is a circular portrait of Thomas Hubert, a man with short dark hair, smiling. Below the portrait, the text 'THOMAS HUBERT' is written in white, followed by the DeepMind logo. On the right side, the text 'UC Berkeley Center for Responsible, Decentralized Intelligence' is at the top in a smaller font. Below it, 'Advanced Large Language Model Agents MOOC' is written in white. A horizontal line separates this from the main title 'AlphaProof: When Reinforcement Learning Meets Formal Mathematics', which is in a larger white font. Another horizontal line follows, and then the date 'March 31st, 2025' is displayed. In the bottom right corner, there is a decorative pattern of white dots.

UC Berkeley Center for Responsible, Decentralized Intelligence

Advanced Large Language Model Agents MOOC

AlphaProof: When Reinforcement Learning Meets Formal Mathematics

March 31st, 2025

THOMAS HUBERT

DeepMind

课程页: Berkeley RDI SP25 course page

录播: YouTube recording

slides: alphaproof.pdf

补充 readings: AlphaZero / The Future of Mathematics?

目录

1 本讲学习目标	2
2 背景与问题设置	2
2.1 为什么 advanced agents 课程会讲 formal mathematics	2
2.2 从数学史到形式化：为什么 formal mathematics 不是怪异小众工具	3
2.3 但为什么 formal mathematics 直到现在都没有普及	3
3 核心概念与术语	4
4 主体讲解：为什么 AlphaZero 可以迁移到数学	4
4.1 先把数学看成一个环境，而不是一堆题目	4
4.2 AlphaZero recipe 的四个 ingredient	5
4.3 AlphaProof 的 foundational bet：少数据但可验证，值得吗	6
5 IMO 2024：为什么这是 benchmark，也是系统工程实战	7
5.1 Apollo-style milestone，而不是“数学 AGI 已成”	7
5.2 比赛期间的 protocol：formalization 与 proving 明确分层	8
5.3 结果、反例与边界条件	9
6 AlphaProof 的系统分解：formalizer、prover、search 与 test-time RL	9
6.1 formalizer：把自然语言数学送进 Lean	9
6.2 prover + proof search：在 Lean state/action 空间里行动	10
6.3 AlphaZero-style RL：不只监督模仿，还要从搜索经验里学习	11
6.4 test-time RL：围绕超难目标问题做局部 specialization	11
7 关键论文与课程 readings 的连接	12
7.1 AlphaZero reading：迁移的是 recipe，不是棋盘	12
7.2 The Future of Mathematics?: 生态位与基础设施视角	13
7.3 把两篇 readings 合在一起看	13
8 例子、失败模式和边界条件	13
8.1 Formal mathematics 的失败模式	13
8.2 与 verification 的关系	13
9 与前后讲的联系	13
10 本章小结	14
11 复习题	14
12 深入思考题	14
13 延伸阅读	14

1 本讲学习目标

本讲回答的是一个比“IMO 上拿了多少分”更深的问题：**为什么形式数学 (formal mathematics) 可能成为高级 LLM agent 最理想的长期环境之一？**读完本讲后，读者应当能回答：

- informal reasoning、formal specification、verification、theorem proving、proof search 之间到底有什么边界。
- 为什么 Lean 能把数学题变成 agent 可交互的环境，而不只是一个排版不同的证明语言。
- AlphaZero 的哪些 ingredient 被 AlphaProof 继承，哪些没有。
- 为什么 perfect verification 很有吸引力，但 formalization bottleneck 仍是决定性瓶颈。
- test-time RL 在 theorem proving 里是什么意思，它与普通采样或 beam search 有什么区别。

2 背景与问题设置

2.1 为什么 advanced agents 课程会讲 formal mathematics

讲者一开始并没有直接谈模型，而是先谈数学的历史位置。原因是：数学既要求**推理与规划 (reasoning and planning)**，又要求**抽象与泛化 (abstraction and generalisation)**，同时还具备开放世界复杂度。很多 agent benchmark 都是在封闭环境里测“会不会操作”，而数学更像是在测“能不能持续构造和验证新知识”。

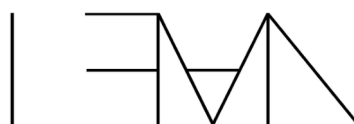
Lean

Lean is a programming language,
theorem prover, and interactive proof
assistant.

It has a [vibrant open source community](#)
of mathematicians.

Lean has been used to formalize [fields](#)
[medal mathematics](#).

A huge quest the community is on it to
build the [math library](#) of the future.



Programming Language and
Theorem Prover

10

图 1: Lean 的统一角色：编程语言、定理证明器和交互式 proof assistant。

这也是本讲和前几讲的连接点。L01 讨论的是 inference-time computation 如何改善 reasoning；L08 进一步指出：如果没有一个能提供可靠反馈的环境，推理再长也很难被真正检查。形式数学的重要性恰恰在于，它把“我觉得这个证明对”变成“proof checker 能否接受这条状态转移链”。

2.2 从数学史到形式化：为什么 formal mathematics 不是怪异小众工具

Slides 用几页历史材料提醒读者：数学本身就是一门不断走向更高符号化、形式化、可传递性的学科。从古希腊对证明的重视，到代数符号逐步取代长篇自然语言，再到现代 proof assistant 的出现，形式化并不是偏离数学，而是把数学中最核心的“可检查性”推到机器层面。

Computer Formalisation unlocks enormous synergies

Perfect Verification + Software Stack

Instant Wins

- Correctness concerns disappear
- Giant Proofs can be trusted
- Proof checking can be delegated entirely to the computer

Game Changer

- Transforms mathematics into a video game 🎮!
- For Education
- All the tools of Software Engineering for Maths
- Unlocks Massive Collaboration

A pilot project in universal algebra to explore new ways to collaborate and use machine assistance?

25 December 2024 in math.AG, preprint | Top | Artificial Intelligence, Equational Theory Project, machine assisted proof, universal algebra | 10 Views 0e

Traditionally, mathematics research projects are conducted by a small number (typically one to five) of expert mathematicians, each of which are familiar enough with all aspects of the project that they can verify each other's contributions. It has been challenging to organize mathematical projects at larger scales, and particularly those that involve contributions from the general public, due to the need to verify all of the contributions; a single error in one component of a mathematical argument could invalidate the entire project. Furthermore, the sophistication of a typical math project is such that it would not be realistic to expect a member of the public, with say an undergraduate level of mathematics education, to contribute in a meaningful way to many such projects.

For related reasons, it is also challenging to incorporate assistance from modern AI tools into a research project, as these tools can "hallucinate" plausible-looking, but nonsensical arguments, which therefore need additional verification before they could be added into the project.

Proof assistant languages, such as Lean, provide a potential way to overcome these obstacles, and allow for large-scale collaborations involving professional mathematicians, the broader public, and/or AI tools to all contribute to a complex project, provided that it can be broken up in a modular fashion into smaller pieces that can be attacked without necessarily understanding all aspects of the project as a whole. Projects to formalize an existing mathematical result (such as the formalization of the recent proof of the P vs NP conjecture of Haken, discussed in this previous blog post) are currently the main examples of such large-scale collaborations that are enabled via proof assistants. At present, these formalizations are mostly crowdsourced by human contributors (which include both professional mathematicians and interested members of the general public), but there are also some recent efforts to incorporate more automated tools (either "good old-fashioned" automated theorem provers, or more modern AI-based tools) to assist with the (often quite tedious) task of formalization.

However, I believe that this sort of paradigm can also be used to explore new mathematics, as opposed to formalizing existing mathematics. The online collaborative "PolyMath" projects that several people including myself organized in the past are one example of this; but as they did not incorporate proof assistants into the workflow, the contributions had to be managed and verified by the human moderators of the project, which was quite a time-consuming responsibility, and one which limited the ability to scale these projects up further. But I am hoping that the addition of proof assistants will remove this bottleneck.

12

图 2: 形式化带来的协同效应：验证、复用、自动化和软件栈共同作用。

讲者把 formalization 的收益总结为几类。第一，**rigor and clarity**：很多在人类交流中默认跳过的步骤，到了 proof assistant 里必须补齐。第二，**efficiency and communication**：一旦某个 lemma 进入库，它就不再只是论文中的文字，而是可重用的软件对象。第三，**unification**：不同领域的证明可以在统一的 formal language 中组合。

不要把 formalization 理解成“多打一遍字”

formalization 的价值不在于把自然语言拷贝成另一种语法，而在于把证明过程变成可执行、可组合、可自动检查的计算对象。对 agent 而言，这意味着状态、动作和反馈第一次被严密地固定了下来。

2.3 但为什么 formal mathematics 直到现在都没有普及

这也是 lecture 非常诚实的一点：讲者没有把 Lean 说成“已经成熟得足以接管数学研究”。Slides 明确列出 adoption 障碍：学习曲线陡峭，formalization 时间投入巨大，Mathlib 的覆盖并不均匀，很多研究数学对象仍缺乏良好库支持。

Challenges for Computer Formalisation

Only 0.1%-1% of mathematicians have adopted Lean.

Not yet a good tool for mainstream research

- Steep Learning Curve
- Significant Time Investment
- Tooling and Library maturity (though rapidly growing)
- Perceived lack of necessity for research.

13

图 3: Formal mathematics 的现实成本: 学习门槛、时间投入与库覆盖。

这正是本讲后续所有技术设计的前提。只要 formalization 仍然昂贵, “能否求证”和“能否把题目表示出来”就是两件不同的事。后者是 **formal specification** 或 **autoformalization** 问题, 前者才是 **theorem proving** 或 **proof search** 问题。二者都依赖 verification, 但不能混为一谈。

3 核心概念与术语

- **informal reasoning**: 自然语言里的推理草图、启发式解释或数学直觉, 通常允许省略步骤。
- **formal specification**: 把问题陈述翻译成 Lean/Isabelle 中 machine-checkable 的定理陈述。
- **verification**: 给定 formal statement 和 candidate proof, 让 proof checker 判断它是否合法。
- **theorem proving**: 在 formal system 内搜索一条满足 checker 的证明。
- **proof search**: 对 proof states、tactics 或 proof tree 进行系统探索的算法过程, 可以是 sampling、beam search、MCTS 或 RL-guided search。
- **autoformalization**: 把自然语言数学自动翻译成 formal theorem 或 formal proof; 这是 theorem proving 的前置步骤之一, 但不是同一个问题。

本讲最关键的区分

verification 负责“验”; theorem proving 负责“找”; autoformalization 负责“写成可验的形式”。一个系统即便 verification 完美, 如果 formalization 很差, 也仍然无法把真实数学问题稳定送进 proving 环节。

4 主体讲解: 为什么 AlphaZero 可以迁移到数学

4.1 先把数学看成一个环境, 而不是一堆题目

Slides 在中段先回顾强化学习的标准四元组: 状态、动作、观察、奖励。这个回顾的目的, 是为了把 proof assistant 重新解释为一个**可交互环境**。在 Lean 里, 当前 proof goal、上下文假设和局部变量

共同组成状态；应用某个 tactic 就是动作；proof state 是否前进、是否报错，就是环境反馈。

What's RL

RL is trial and error learning.

An **agent** learns by **interacting** with an **environment** to maximize **reward**.

24

图 4: 把 proof assistant 重述成 RL 环境。

如果把 prover 策略记为 π ，那么最粗粒度的目标可写成

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T r_t \right].$$

这里 τ 是一条 proof-search 轨迹， r_t 来自 Lean checker 或相关辅助模块给出的 grounded signal。这个式子当然没有捕捉 theorem proving 的全部结构，但它把核心直觉说清楚了：**proof search 不是散文创作，而是带有可检查状态转移的 sequential decision-making。**

4.2 AlphaZero recipe 的四个 ingredient

讲者随后回顾 Alpha 系列系统的共同 recipe: **scaled up trial and error**、**grounded feedback**、**search**、**curriculum**。把这四项映射到数学上，会得到非常具体的工程判断。

What made those systems Superhuman?

Scaled up trial and error

Grounded feedback signal

Search

Curriculum

30

图 5: Alpha 系列 recipe 的四个 ingredient。

- **trial and error**: 不断尝试 tactics、构造 proof branches、回溯失败路径。
- **grounded feedback**: Lean checker 对每一步 state transition 给出精确反馈，不靠人工偏好打分。
- **search**: 不能只依赖单步 greedy tactic generation，而要在 proof tree 上做更全局的探索。
- **curriculum**: 先在已有 formal corpus 与较容易问题上学会 action prior，再逐步进到更难定理。

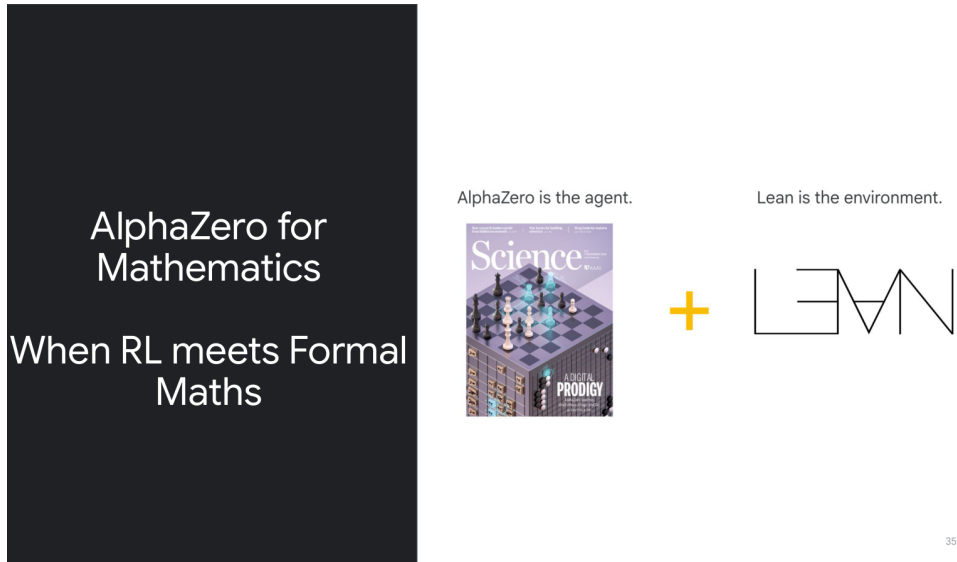


图 6: “AlphaZero for Mathematics” 是本讲的中心命题。

如果采用 AlphaZero 风格的先验引导搜索，动作选择可抽象成

$$a^* = \arg \max_a \left(Q(s, a) + c P_\theta(a | s) \frac{\sqrt{N(s)}}{1 + N(s, a)} \right).$$

这里 s 是当前 Lean proof state, a 是候选 tactic, $Q(s, a)$ 是 value estimate, $P_\theta(a | s)$ 是 prover 模型提供的动作先验。直觉上，搜索既不能完全听模型先验，也不能盲目穷举；它要在高价值路径利用与低访问路径探索之间平衡。

```

1 while budget remains:
2     sample candidate tactics from the prover prior P(a|s)
3     expand/search promising states with value-guided tree search
4     apply a tactic inside Lean
5     observe the next proof state and checker feedback
6     backpropagate success/failure signals

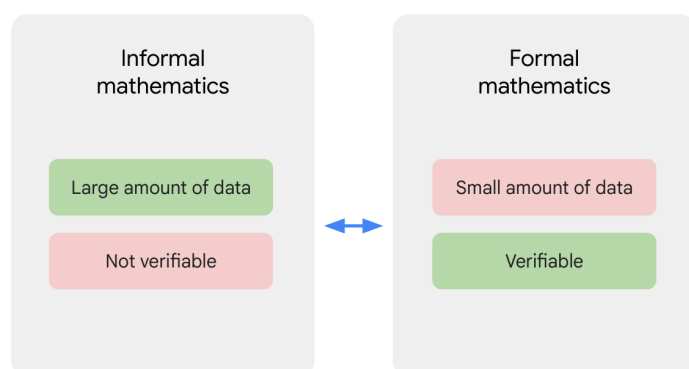
```

为什么朴素 prompting 不够 若只让模型“直接写完整证明”，它其实是在巨大 action space 中一次性押注整条轨迹。任何一步不合法都会让整条轨迹失效。Proof search 的必要性在于：**数学证明的中间状态非常重要，且可以被环境检查。**

4.3 AlphaProof 的 foundational bet: 少数据但可验证，值得吗

这是本讲最有洞见的判断。Slides 明确写出：informal mathematics 拥有大量数据，但不可验证；formal mathematics 拥有较少数据，但可验证。

Let's take a step back...

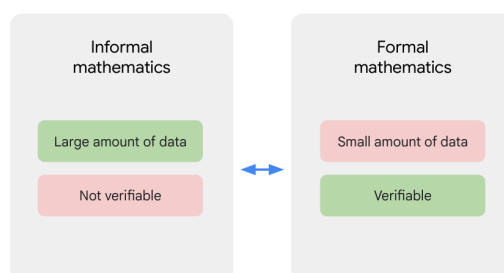


40

图 7: 数据量与可验证性的张力。

AlphaProof: Foundational Bet

Perfect verification will in the long run be the most important property for mathematics.



41

图 8: “Perfect verification” 是讲者的 foundational bet。

这意味着 AlphaProof 的路线不是去追逐最大数据规模，而是在**验证信号质量**上押注。理由是：长期看，只要 verifier 足够可靠，trial-and-error search 就能被持续扩展；而如果反馈本身不可信，再长的 CoT 和再多的 sampled proofs 也只是不可检验文本。这个判断和前几讲中的一个 recurring theme 是一致的：**高质量外部反馈比纯自说自话更关键**。

5 IMO 2024: 为什么这是 benchmark，也是系统工程实战

5.1 Apollo-style milestone，而不是“数学 AGI 已成”

讲者把 IMO 2024 比作 Apollo program，并不是说解出 IMO 就等于通用数学家诞生，而是说：在算力、时间和软件栈都受限的现实条件下，能否让系统在一个具有全球可比性的 benchmark 上完成高难度任务。

Our IMO Participation - Apollo program

Can we reach the moon?

Can our system solve the 2024 IMO problems at all, given

- the compute available to us.
- and enough time.



54

图 9: IMO 2024 作为 Apollo-style milestone。

这一 framing 很重要，因为它重新定义了成功标准。系统不是要证明“数学已经 solved”，而是要证明 **formal verification + search + RL** 这条路线具有外部可见的里程碑价值。

5.2 比赛期间的 protocol: formalization 与 proving 明确分层

比赛流程页揭示了一个非常重要的工程事实：**AlphaProof** 并不是端到端从自然语言 IMO 题目直接出 **formal proof**。真正的 protocol 包括：专家先将题目 formalize 到 Lean；Gemini 生成大量候选答案；系统用 disprover 去剔除明显错误的猜测；再由 AlphaProof/AlphaGeometry 对剩余候选进行深搜。

Our Protocol

After officially receiving the problems at 1PM,

1. Lean experts manually formalize problems
2. Generate O(100) answer candidates with Gemini
3. Filter the easily disprovable ones
4. Run test-time RL



图 10: 比赛日 protocol: manual formalization、candidate generation、disproving 与 proof search 的分工。

```

2 manual Lean formalization by experts
3 generate 0(100) candidate answers with Gemini
4 use disprover to eliminate easy wrong guesses
5 run AlphaProof/AlphaGeometry on the remaining candidates and proofs

```

这个 protocol 之所以值得细讲，是因为它把 **formal specification** 与 **theorem proving** 强行拆开。前者仍依赖专家；后者才是 AlphaProof 真正擅长的部分。这既是系统成功的原因，也是当前能力边界最诚实的描述。

5.3 结果、反例与边界条件

Final Results

P1, P2, P6 fully solved by AlphaProof
P4 fully solved by AlphaGeometry

We keep running over the weekend in
the hope of getting one point on P3.
The agent made some progress but
not enough for a partial point.



图 11: AlphaProof 与 AlphaGeometry 在 IMO 2024 的结果。

Slides 展示的结果非常强，但更值得注意的是失败分布。P4 几何题需要 AlphaGeometry 才能补上，P3/P5 的组合与几何难点暴露出 Mathlib 覆盖、formalization 难度和 proof-search branching factor 的多重瓶颈。也就是说，**验证信号完美并没有自动消除表示、建模和搜索的困难。**

不要把银牌结果误读为“形式数学已被攻克”

AlphaProof 证明的是：在有 formal environment、强搜索和足够工程投入时，系统能在高难 benchmark 上取得可信成绩。它没有证明：自然语言数学题可以被无缝 formalize，或者开放研究数学已经接近 solved。

6 AlphaProof 的系统分解：formalizer、prover、search 与 test-time RL

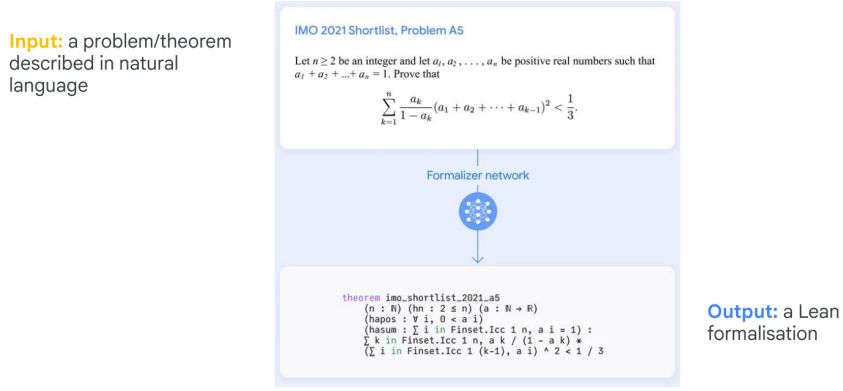
6.1 formalizer：把自然语言数学送进 Lean

Slides 在后半段明确给出 formalizer 接口：输入是自然语言问题或定理，输出是 Lean 中的 formal statement。

$$\hat{T}_{\text{formal}} = f_{\phi}(x_{\text{informal}}).$$

其中 x_{informal} 是自然语言题目, f_ϕ 是 formalizer, $\hat{T}_{\text{formal}}$ 是转写后的 formal theorem statement。

Formalizer Model



78

图 12: Formalizer 的输入输出接口。

这一步不等于 proving。它只是在构造“什么才是需要被证明的对象”。为什么这一步困难？因为自然语言问题包含缩写、隐含量词、默认背景知识，有时甚至包含图示或约定俗成的数学写法。形式系统无法容忍这些省略。

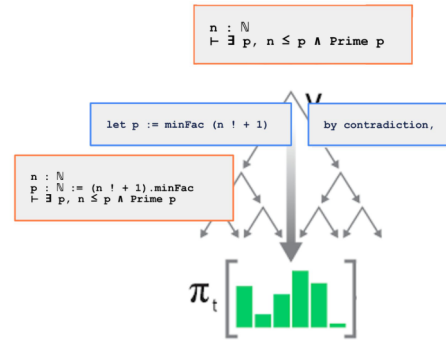
6.2 prover + proof search: 在 Lean state/action 空间里行动

Prover Model + AlphaZero Search

Search over **actions** = Lean tactic

Compute new **Lean state** after every **action / tactic** application

Exploit high prior and high value paths
Explore low visited paths



80

图 13: Prover 在 Lean 中搜索 tactic。

在 theorem proving 阶段, 模型不再输出漂亮散文, 而是输出 tactics 或证明步骤。每一步都会改变 proof state。这个过程与普通自然语言生成最不同的地方在于: **每一步都能被环境立即判定是否合法**。因而 proof search 更接近 game playing 或 program synthesis, 而不是开放式写作。

6.3 AlphaZero-style RL: 不只监督模仿, 还要从搜索经验里学习

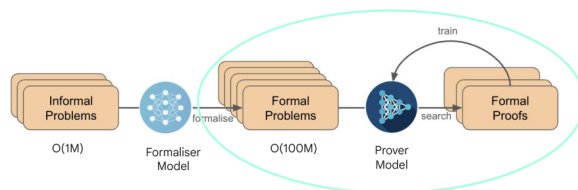
讲者随后给出训练管线: 先用 formalized problems 和 Mathlib 做监督学习, 学到 action prior; 再在 proof-search 环境中收集 proving/disproving experience, 用 RL 继续提升策略。

Step 3: AlphaZero Reinforcement Learning

3: Train the prover model by RL

For each formal problem

- Generate experience of (dis)proving by searching over Lean steps.
- Use Lean to verify proofs
- Reinforce the prover network with each success



84

图 14: AlphaZero-style RL 在 theorem proving 上的经验生成。

```
1 informal problem -> formalizer model -> Lean theorem statement
2 formal theorem state -> prover model + search -> tactic sequence
3 Lean checker verifies every state transition and the final proof
```

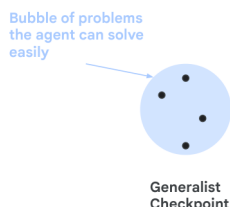
这里的一个重要直觉是: 仅做 imitation learning 会把模型限制在现有人类 proofs 分布附近; 而 RL 让系统能在 search 中发现**人类数据里不存在、但被 checker 认可**的新路径。这也是讲者强调 “discovering knowledge by themselves” 的原因。

6.4 test-time RL: 围绕超难目标问题做局部 specialization

最有 agent flavor 的部分是 test-time RL。Slides 把它画成一个 “problem bubble”: 从 generalist checkpoint 出发, 在目标难题周围生成变体问题, 利用这些局部问题继续 RL, 形成更适合该难题附近分布的 specialist。

Final Step: Test-Time RL

● Very hard problem



86

图 15: 从 generalist 到 specialist 的 test-time RL bubble。

可抽象为

$$\theta' = \arg \max_{\theta} \mathbb{E}_{\tilde{p} \sim \mathcal{N}(p)} [R_{\text{Lean}}(\pi_{\theta}, \tilde{p})].$$

这里 p 是目标难题， $\mathcal{N}(p)$ 是围绕它构造的变体分布， R_{Lean} 是 Lean 反馈定义的 reward。直觉是：如果原始问题太难，直接在它上面 RL 信号稀薄；但若能找到与之邻近的一簇可解变体，就能在局部形成有效 curriculum。

```
1 start from a generalist checkpoint
2 construct variants around the target hard problem
3 search and collect proving/disproving experience
4 update the prover by RL on that local bubble
5 deploy the specialized checkpoint back onto the target problem
```

为什么这不是简单的“多采样一些” 多采样只是增加宽度，test-time RL 则是在**改变策略本身**。它让系统在部署时仍保留学习能力，这一点与本课程“agent 不是静态 predictor，而是带有环境反馈闭环的系统”高度一致。

7 关键论文与课程 readings 的连接

7.1 AlphaZero reading: 迁移的是 recipe，不是棋盘

《Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm》为本讲提供了最核心的算法祖先。Lecture 真正继承的是一种 harness：强先验模型、search、grounded feedback、自生成经验与 iterative improvement 的闭环。不同之处在于，棋类环境天然可模拟，而数学环境需要 Lean/Mathlib 这样的形式系统来构造。

更具体地说，AlphaZero 给 AlphaProof 提供的不是某个棋类专用 trick，而是三条更一般的系统原则。第一，**环境必须给出无歧义反馈**，否则 search 与 RL 无法稳定闭环。第二，**policy/value 与 search 要联动**，单独扩大采样宽度并不等于有效搜索。第三，**经验生成本身是能力来源**，系统要能从自身 rollouts 中不断积累更有用的 proving data。

但这条迁移也有明确断点。棋类里 legal move checker、终局 reward 和 simulator 都是现成的；形式化数学里，formalization 成本、library coverage 和 tactic/action space 远比棋盘复杂。因此 lecture 把 AlphaZero 当作 ancestor，是为了说明 *recipe continuity*，不是为了暗示 theorem proving 已经像棋类那样“环境完备”。

7.2 The Future of Mathematics?: 生态位与基础设施视角

这段 reading 不是 benchmark paper，但它解释了为什么 formal mathematics 社区建设本身重要。没有 Lean 教学、Mathlib 沉淀和 formal abstracts 这样的生态基础，AlphaProof 无法把数学变成 machine-actionable environment。换句话说，**agent 的上限不仅受模型限制，也受环境基础设施成熟度限制。**

这也是本讲和很多“数学 AI demo”最不一样的地方。lecture 并没有把形式化数学当成一个单独 benchmark，而是把它视为一套逐步成熟中的 research infrastructure。Buzzard 那条线强调 curriculum digitization、formal abstracts 和教学生态，其实是在回答一个更底层的问题：**如果环境本身还没有被系统化整理出来，再强的 proving agent 也只能在很窄的子空间里工作。**

7.3 把两篇 readings 合在一起看

把 AlphaZero 和《The Future of Mathematics?》放在一起，能更准确地理解 AlphaProof 的位置。前者解释了为什么 search + policy/value + self-generated data 是强 agent recipe；后者解释了为什么 Lean / Mathlib / formal education community 是使该 recipe 可落地的环境前提。没有前者，AlphaProof 不知道如何在 environment 中学习；没有后者，AlphaProof 连一个可持续扩展的 environment 都没有。

8 例子、失败模式和边界条件

8.1 Formal mathematics 的失败模式

- **representation bottleneck**: 题目还没被 formalize，prover 再强也无从行动。
- **library bottleneck**: Mathlib 缺什么，系统就很难有效进入对应领域。
- **action-space explosion**: tactic 选择空间巨大，局部贪心会迅速陷入死路。
- **benchmark overfitting**: 若一味围绕某类 Olympiad 题做 specialization，系统可能难以转向开放研究数学。

8.2 与 verification 的关系

verification 是整个路线成立的底座，但它只回答“这一步是否合法”。它不回答：哪条 proof path 更容易找到？哪个 formalization 最贴近原题？哪种 curriculum 更值得投入算力？因此 verification 是必要条件，不是充分条件。

9 与前后讲的联系

与 L01 相比，本讲把“需要外部反馈”这一原则推进到了极致：Lean checker 给出的是几乎无歧义的 grounded feedback。与下一讲 L09 相比，本讲更偏向系统和环境视角；L09 会更细分 autoformalization、theorem proving、retrieval、evaluation gaps 等问题，并系统解释为什么 formal mathematics 不是一个单一任务。

10 本章小结

AlphaProof 的重要性，不在于它把一个 benchmark 做到了多高，而在于它展示了 formal mathematics 可以承载高级 agent 的完整闭环：问题表示、状态转移、外部验证、搜索、强化学习和 test-time specialization。它也同样清楚地展示了边界：formalization 成本、Mathlib 覆盖和 proof-search action space 仍然是关键难点。

11 复习题

1. 为什么讲者认为 mathematics 是 intelligence 的 root node?
2. Lean 在 AlphaProof 里分别扮演了哪些角色?
3. AlphaZero recipe 的四个 ingredient 如何映射到 theorem proving?
4. 为什么 perfect verification 依然无法自动解决 formalization bottleneck?
5. test-time RL 与普通多样本采样有什么本质差异?

12 深入思考题

1. 如果未来 autoformalization 大幅进步，AlphaProof 的系统瓶颈会转移到哪里?
2. 在 theorem proving 中，value model、search policy 与 retrieval 分别应承担什么职责?
3. formal mathematics 是否一定比开放世界 web/GUI agent 更适合作为长期 AGI benchmark? 为什么?

13 延伸阅读

- *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*
- *The Future of Mathematics?*
- AlphaGeometry / Mathlib / Lean ecosystem materials